



**TECNOLÓGICO
NACIONAL DE MÉXICO**

INSTITUTO TECNOLÓGICO SUPERIOR DE JEREZ

Carrera: Ingeniería en Sistemas Computacionales

Nombre de la Materia: Tópicos Selectos Web

**Nombre de la Actividad: Reporte de Practica (Indexed
DB)**

Nombre del Alumno(a): Nayeli Morales García

Número de Control: S19070105

Semestre: 14

Nombre del Docente: Salvador Acevedo Sandoval

Fecha de Entrega: miércoles 06 de mayo del 2026

Introducción

IndexedDB es una API del navegador que permite almacenar grandes cantidades de datos de forma estructurada directamente en el cliente, sin necesidad de conexión a un servidor. A diferencia de otras opciones de almacenamiento como LocalStorage o SessionStorage, IndexedDB está diseñada para manejar volúmenes de información más complejos y grandes, ya que utiliza una base de datos orientada a objetos en la que los datos se guardan en “almacenes” (object stores) en lugar de tablas tradicionales. Esta característica la hace especialmente útil para aplicaciones web modernas que requieren funcionamiento offline, almacenamiento de datos persistente o manejo de información compleja como listas de usuarios, catálogos o configuraciones personalizadas.

El funcionamiento de IndexedDB es asíncrono, lo que significa que las operaciones de lectura y escritura no bloquean la ejecución del resto del código en la aplicación. Esto se logra mediante el uso de eventos y solicitudes (requests), como abrir una base de datos, crear almacenes o realizar transacciones. Cada operación se ejecuta dentro de una transacción, lo cual garantiza la integridad de los datos y permite manejar errores de forma controlada. Además, IndexedDB permite el uso de índices para optimizar búsquedas y consultas, lo que mejora significativamente el rendimiento en aplicaciones que manejan grandes volúmenes de información.

Otra característica importante es que IndexedDB permite almacenar distintos tipos de datos, incluyendo objetos JavaScript completos, archivos e incluso datos binarios. Esto la convierte en una herramienta muy versátil para desarrolladores que necesitan persistencia avanzada en el navegador. Sin embargo, su uso puede resultar más complejo en comparación con otras soluciones de almacenamiento debido a su naturaleza basada en eventos y a la gestión de versiones de la base de datos, la cual se realiza mediante el evento “onupgradeneeded”.

En el contexto de una práctica, IndexedDB resulta fundamental para comprender cómo las aplicaciones web pueden gestionar datos de manera local, eficiente y segura. Su implementación permite desarrollar aplicaciones más robustas, capaces de funcionar incluso sin conexión a internet, mejorando la experiencia del usuario y ampliando las posibilidades del desarrollo web moderno.

Desarrollo

Abrimos el espacio o carpeta donde están los talleres y creamos la carpeta del taller.



Ahora abrimos el cmd e ingresamos a la carpeta que creamos del taller. Y enseguida inicializamos el proyecto que se crean los package.json y los modulos de node.

```
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres> cd indexDB
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\IndexDB> npm init -y
Wrote to C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\IndexDB\package.json:

{
  "name": "indexdb",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Luego de esto instalamos express ya que trabajaremos con él.

```
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\IndexDB> npm install express

added 65 packages, and audited 66 packages in 34s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\IndexDB>
```

Ya teniendo eso, en la carpeta raíz del proyecto, creamos el archivo server.js e ingresamos el código que tiene en la siguiente imagen que es para que funcione o se ejecute el servidor en el puerto que decidimos.

A screenshot of a code editor with a dark theme. The top bar shows a tab for 'server.js' with a yellow 'JS' icon and a close button. The editor content shows a command prompt 'IndexDB >' followed by 'server.js > ...'. Below this, there is a list of 9 lines of JavaScript code. Line 1: 'const express = require('express');'. Line 2: 'const app = express();'. Line 3: 'const path = require('path');'. Line 4: an empty line. Line 5: 'app.use(express.static('public'))';'. Line 6: an empty line. Line 7: 'app.listen(3000, () => {''. Line 8: ' console.log("Servidor escuchando en http://localhost:3000");'. Line 9: '});'. The code is color-coded: 'const' is blue, 'express' is green, 'require' is red, 'path' is green, 'app' is blue, 'static' is green, 'public' is red, 'listen' is blue, '3000' is red, '=>' is blue, 'console.log' is red, and the string content is white. A red dot is visible in the left margin next to line 2.

```
server.js X
IndexDB > server.js > ...
1  const express = require('express');
2  const app = express();
3  const path = require('path');
4
5  app.use(express.static('public'));
6
7  app.listen(3000, () => {
8    console.log("Servidor escuchando en http://localhost:3000");
9  });
```

Ahora en la carpeta raíz del proyecto, creamos la carpeta 'public' y dentro de ella creamos lo que es un index.html le damos formato de html 5, luego cambiamos el título de la página y le ponemos Indexed DB. Dentro del body comenzamos a agregar código. Ponemos un título y creamos una tabla con unos botones para ver el funcionamiento de esta práctica.

```

IndexDB > public > index.html > ...
1 |
2 <!DOCTYPE html>
3 <html lang="es">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>IndexedDB</title>
8 </head>
9 <body>
10
11   <h1>Base de Datos Local</h1>
12
13   <button onclick="conectarDB()">Conectar a IndexedDB</button>
14   <button onclick="guardarUsuario()">Registrar Usuario</button>
15   <button onclick="consultar()">Cargar Tabla de Usuario</button>
16
17   <br><br>
18
19   <table border="1">
20     <thead>
21       <tr>
22         <th>ID</th>
23         <th>Nombre</th>
24         <th>Sexo</th>
25         <th>Acciones</th>
26       </tr>
27     </thead>
28     <tbody id="cuerpo-tabla"></tbody>
29   </table>
30

```

Ahora terminando la tabla mismo dentro del body creamos un script y hacemos la variable de la base de datos y creamos la función para conectar la base de datos, ya que un botón que creamos es para darle click y que se conecte a la base de datos si es así muestra el conectado a la base de datos y si no pues muestra el error de que no se pudo conectar.

```

<script>
  let db;

  function conectarDB() {
    const req = indexedDB.open("MiBaseLocal", 1);

    req.onsuccess = (e) => {
      db = e.target.result;
      alert("Base de datos conectada correctamente");
    };

    req.onupgradeneeded = (e) => {
      const dbUpgrade = e.target.result;

      if (!dbUpgrade.objectStoreNames.contains("usuarios")) {
        dbUpgrade.createObjectStore("usuarios", { keyPath: "id" });
        console.log("Almacén 'usuarios' creado");
      }
    };

    req.onerror = (e) => {
      console.error("Error al abrir la base de datos", e.target.error);
    };
  }

```

Ahora creamos la función para guardar el usuario. En este pues verifica que si funcione para la conexión de la base de datos y pues ahí mismo agregamos un usuario. Las variables que agregamos pues les agregamos valores o texto para usarlas en la tabla que creamos al principio y pues agregamos código que se mostrara si se agregó o hubo algún error en el usuario y que se mostraran alertas.

```

function guardarUsuario() {
  if (!db) {
    alert("Primero debes conectar la base de datos");
    return;
  }

  try {
    const transaccion = db.transaction(["usuarios"], "readwrite");
    const almacen = transaccion.objectStore("usuarios");

    const nuevoUsuario = {
      id: Date.now(),
      nombre: "Yo",
      sexo: "F",
      tema: "Obscuro",
      fecha_registro: "2026-05-06"
    };

    const petition = almacen.add(nuevoUsuario);

    petition.onsuccess = () => {
      alert("Usuario registrado exitosamente");
      consultar();
    };

    petition.onerror = (e) => {
      console.error("Error al guardar:", e.target.error);
    };
  } catch (error) {
    console.error("Error en guardarUsuario:", error);
  }
}

```

Luego agregamos otra función que se llama consultar, que pues verifica que si haya conexión con la base de datos y se muestren los datos de el usuario que se acaba de registrar. Y esto pues aparece en una tabla que se crea en el HTML que se manda llamar la que se creo al inicio, pero en HTML y también puedes eliminar el registro del usuario. En caso de que haya error pues se muestre cual fue el motivo que no se hizo la acción seleccionada.

```

function consultar() {
  if (!db) {
    alert("Primero conecta la base de datos");
    return;
  }

  const tabla = document.getElementById("cuerpo-tabla");
  tabla.innerHTML = "";

  const transaccion = db.transaction(["usuarios"], "readonly");
  const almacen = transaccion.objectStore("usuarios");
  const cursorRequest = almacen.openCursor();

  cursorRequest.onsuccess = (e) => {
    const cursor = e.target.result;

    if (cursor) {
      const u = cursor.value;

      tabla.innerHTML += `
        <tr>
          <td>${u.id}</td>
          <td>${u.nombre}</td>
          <td>${u.sexo}</td>
          <td>
            <button onclick="eliminarRegistro(${u.id})">Eliminar</button>
          </td>
        </tr>
      `;

      cursor.continue();
    }
  };

  cursorRequest.onerror = (e) => {
    console.error("Error al consultar:", e.target.error);
  };
}

```

Por último, se crea la función para eliminar el registro. Que pues este lo que hace es que verifica los usuarios en la base de datos donde se almacenan y los borra por el id para saber cual usuario se esta borrando desde la tabla que se muestra pero en la base de datos le asigna un id como identificador. Si no se puede borrar pues sale el error.

```

function eliminarRegistro(idABorrar) {
  if (!confirm("¿Seguro que quieres eliminar el ID " + idABorrar + " ?")) return;

  const transaccion = db.transaction(["usuarios"], "readwrite");
  const almacen = transaccion.objectStore("usuarios");

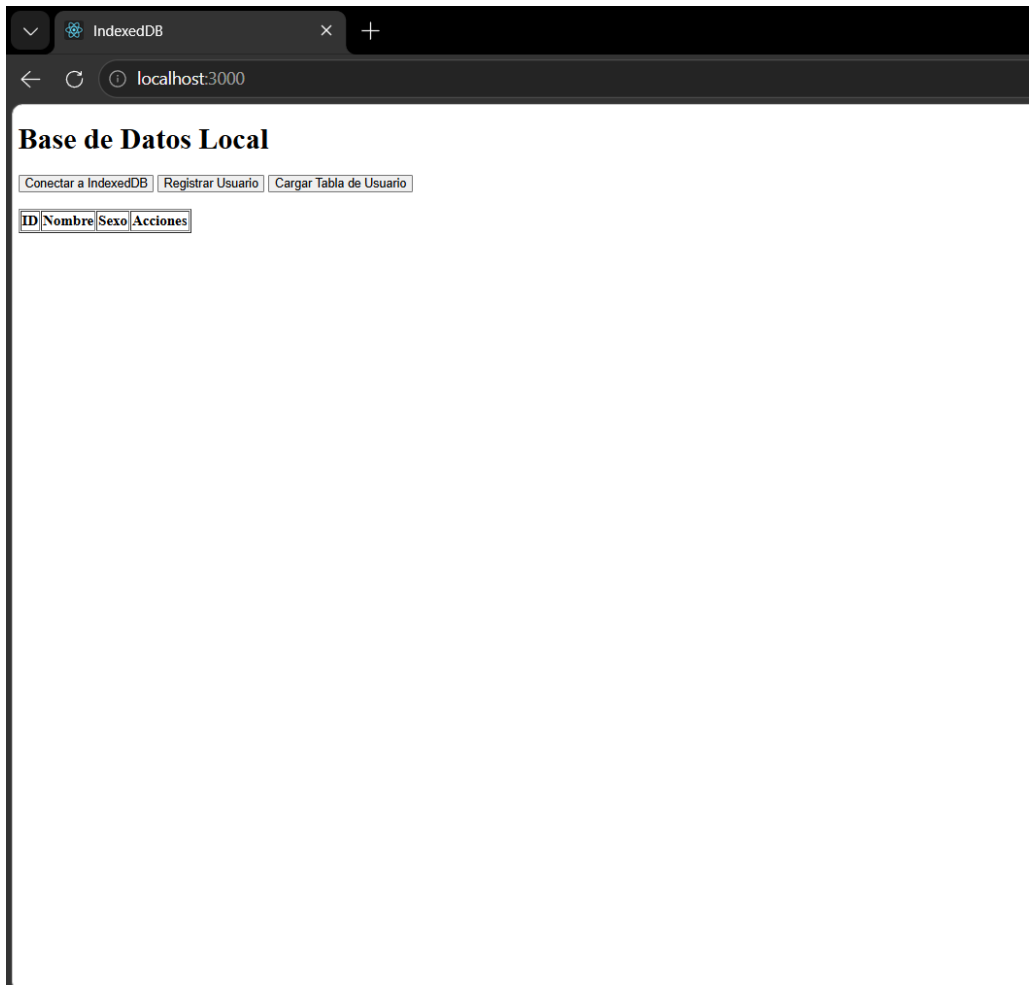
  const petition = almacen.delete(idABorrar);

  petition.onsuccess = () => {
    alert("Registro " + idABorrar + " eliminado");
    consultar();
  };

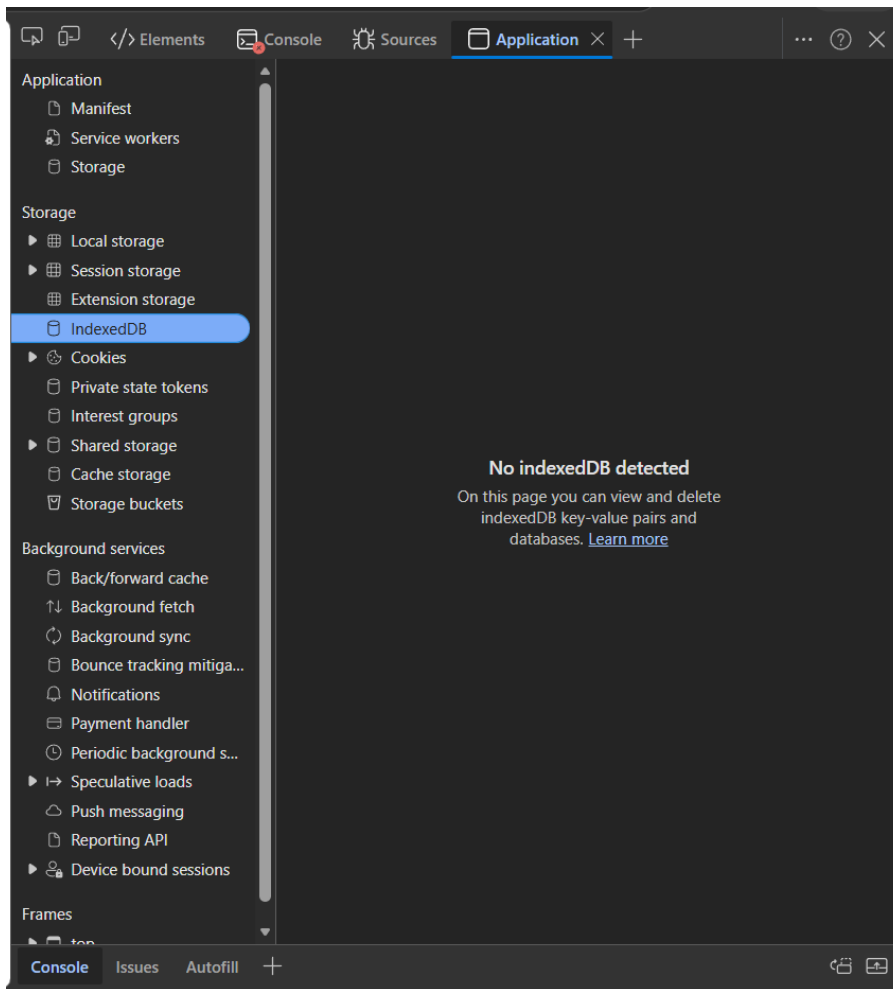
  petition.onerror = (e) => {
    console.error("No se pudo eliminar:", e.target.error);
  };
}
</script>
</body>
</html>

```

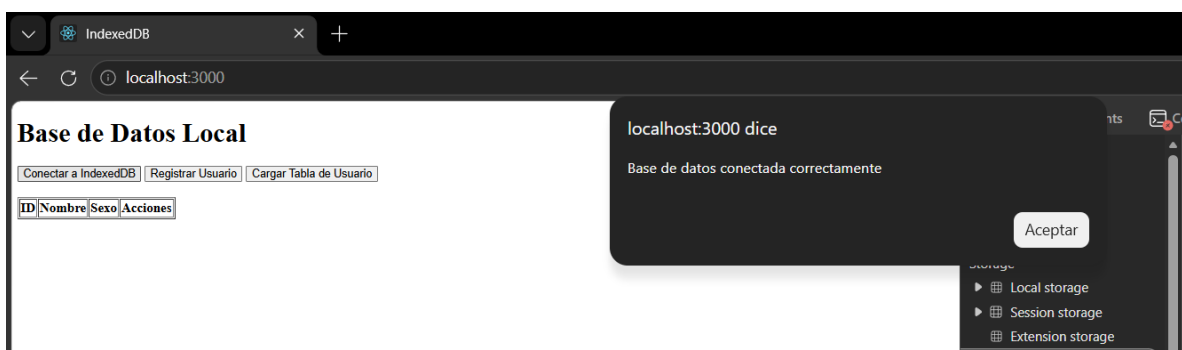

Ya teniendo todo el código completo, abrimos la terminal, nos aseguramos que estemos dentro de la carpeta de nuestro proyecto y vamos a correr el servidor con el comando `server.js` y se mostrara si todo esta bien el mensaje que ingresaste y el puerto o dirección donde esta funcionando. Abrimos esa dirección y nos aparecerá como en la imagen. Claro, si no hay ningún error.



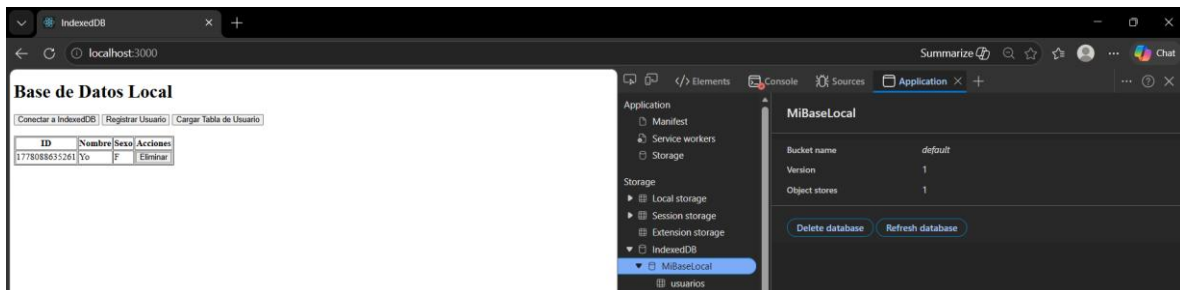
Ya que todo esta bien nos muestra la tabla y los botones que creamos, con todo y su función que esa no se visualiza. En cualquier lado de la pagina damos click derecho y click en inspeccionar nos aparece un menú en la barra lateral derecha y buscamos la opción application le damos click y en el menú del lado izquierdo que aparece buscamos IndexedDB y nos aparece que no a detectado ninguna base de datos como se muestra en la imagen de aquí en seguida.



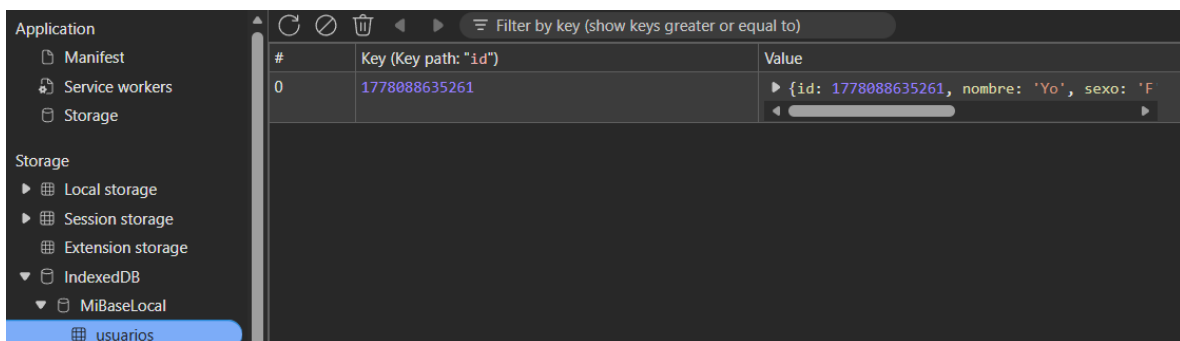
Ahora probamos lo que es el botón de conectar pues a la base de datos y vemos en la imagen de aquí abajo que si está funcionando.



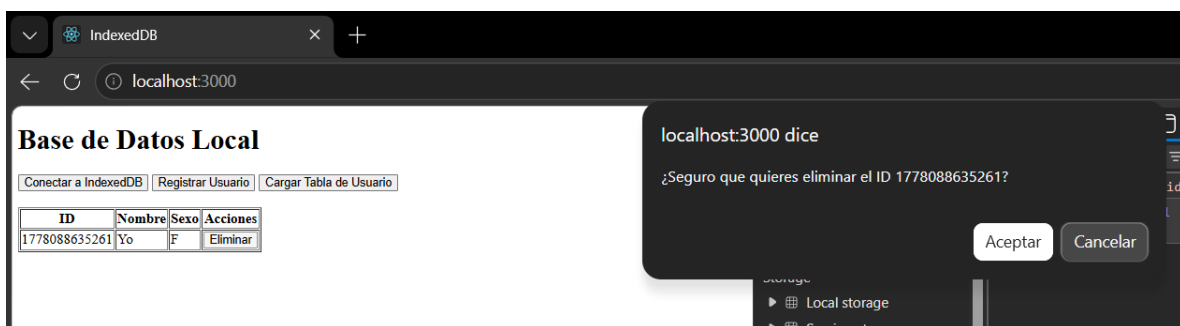
Después le damos en registrar usuario que se deben de cargar los datos que ingresamos en la función de guardar usuario y efectivamente en el apartado de indexed db ya detecto algo que es la base de datos y dicha tabla que creamos como aparece en seguida.



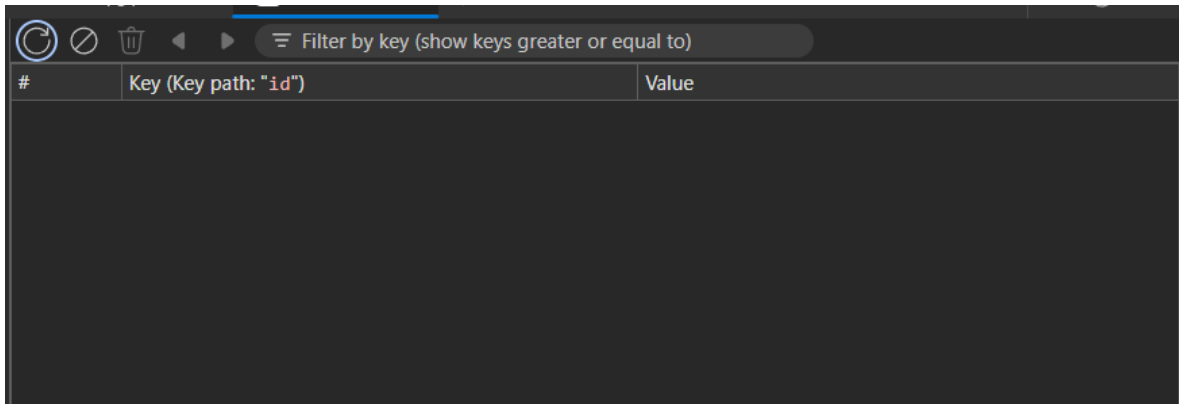
Ahora entramos a la tabla, bueno damos click en la tabla de usuarios que apareció en la base de datos y ahí se va a mostrar los datos del usuario como el ID, Nombre, sexo etc. Todas las variables que estamos utilizando. Se puede ver en la imagen.



Regresando a la tabla pues también nos muestra el usuario que se agregó, para probar el funcionamiento del botón eliminar le damos click y nos va a preguntar que si estamos seguros de hacerlo le damos en aceptar.



Y pues efectivamente se elimino y ya no aparece la información del usuario que se agregó.



Esto quiere decir que la práctica si funciona.

Conclusion

En conclusión, IndexedDB es una herramienta muy importante dentro del desarrollo web moderno, ya que permite almacenar y administrar grandes cantidades de información directamente en el navegador del usuario de manera estructurada y persistente. A través de esta práctica fue posible comprender el funcionamiento de las bases de datos locales en aplicaciones web, así como la creación de almacenes de objetos, el manejo de transacciones y la realización de operaciones básicas como agregar, consultar y eliminar registros.

Además, se observó que IndexedDB ofrece ventajas significativas frente a otros métodos de almacenamiento local, debido a su capacidad para manejar datos complejos y trabajar de forma asíncrona sin afectar el rendimiento de la aplicación. Esto la convierte en una opción ideal para aplicaciones que requieren funcionamiento offline o almacenamiento de información en el lado del cliente.

Por otra parte, la práctica permitió reforzar conocimientos de JavaScript relacionados con eventos, manipulación del DOM y control de errores, integrando conceptos tanto de programación web como de gestión de datos. En general, el uso de IndexedDB representa una solución eficiente y flexible para el desarrollo de aplicaciones web más dinámicas, rápidas y funcionales, ampliando las posibilidades de almacenamiento y mejorando la experiencia del usuario.